

Vortex GPU: BrainFloat16 Implementation

Udit Subramanya

1 Overview & Background

Over the last few years, heterogeneous computing has grown in popularity and applicability in the high-performance computing space. *ARM*, a semiconductor company, defines heterogeneous computing as a system that utilizes multiple types of computing cores, such as a Central Processing Unit (CPU), Graphics Processing Unit (GPU), Field Programmable Gate Array (FPGA), or Tensor Processing Unit (TPU) [1]. By assigning different workloads to each of these specialized hardware units, performance and energy efficiency are improved. In particular, machine learning and artificial intelligence workloads can be accelerated with the use of a GPU co-processing alongside a CPU. *NVIDIA*, a market-leading GPU manufacturer, explains that GPUs break complex problems into thousands or millions of smaller repetitive computations and rely on their massively parallel architecture to evaluate them all at once [2]. Despite this, there are still numerous optimizations that could be realized on GPUs.

Fundamentally, computers only understand binary representations, so all data must be encoded in ones and zeros. This occurs naturally for whole numbers by using a base-2 numbering system. A challenge arises when trying to represent decimal numbers, more formally known as floating-point (FP) numbers. In 1985, the *Institute of Electrical and Electronics Engineers* defined a floating-point standard (IEEE-754) for interpreting 32 bits of binary data as an FP number. Since then, IEEE-754 has become the de facto standard on all computers. Due to the intricacy of IEEE-754, complex hardware is required within a GPU to complete floating-point arithmetic operations [3].

Google Brain recently introduced BrainFloat16 (BF16), a novel FP standard for ML and AI acceleration. BF16 utilizes 16 bits, providing greater efficiency in storage and computation while still maintaining accuracy in ML and AI workloads [4]. The adoption of BF16 in hardware accelerators greatly enhances performance and efficiency, particularly in power-intensive data center environments. This cost-effective approach improves complex AI and ML computations significantly.

2 Proposal

I propose contributing a working BF16 implementation to the open source Vortex GPU project and conducting performance benchmarking. The Vortex research project, led by Professor Hyesoon Kim at Georgia Tech, implements a functional GPU on FPGA hardware. The re-programmable nature of FPGAs allows for the hardware components to be modified making it a suitable platform for research and testing.

Additionally, the extensible design of the Vortex source code facilitates the integration of new features, including new BF16 hardware units that I aim to add. I plan to get guidance from Varun Saxena, Professor Kim's graduate student, who is an experienced developer on the Vortex platform. The Vortex research lab already possesses the necessary hardware and infrastructure, which eliminates any capital expenses for my completion of this project.

I have a particular affinity towards this research project given my exposure the prerequisite knowledge required to carry out this research project. From a computer hardware perspective, I was a CS 2110 teaching assistant for four semesters. The class teaches numerous computer hardware principles, specifically an entire unit covering IEEE-754. From a programming language and development standpoint, I was a student in Professor Kim's class where I learned about FPGA development and GPU architectures through a series of hands on projects.

3 Approach & Methodology

I plan to split the software development into several phases. Each phase marks a critical milestone towards completing this research project, and aim to be specific enough to keep me on track, but broad enough to account for new findings along the software development process.

3.1 Phase 1: Understanding BF16 Datatype

I'll study the BF16 standard, by examining its similarities with IEEE-754, and consult implementations from linear algebra and machine learning libraries like Eigen and PyTorch. With this knowledge, I expect to know how to convert between floating point numbers and BF16 manually.

3.2 Phase 2: Basic Arithmetic Operations

Next, I need to determine how to perform basic arithmetic operations between two BF16 representations, as my ultimate goal is to accelerate FP calculations on a GPU. Once basic arithmetic operations are established, complex operations can be built as a hybrid of the basics. To gain insight into the internal workings of floating point arithmetic, I can refer to IEEE-754 hardware implementations [3]. Additionally, I must address edge cases, such as when a numerical result is infinity or not-a-number. At this point, I should be able to produce pseudocode for a basic BF16 library.

3.3 Phase 3: SimX Verification

SimX is a tool in Vortex that allows developers to rapidly prototype their code before implementing it on hardware. So, I plan to utilize SimX as a testing platform for my BF16 implementation in C++ code. To verify the successful completion of this phase, I will write test cases to check the accuracy of my BF16 implementation. For example, I could test the result of $\frac{1}{3} + \frac{21}{7}$ against the IEEE-754 standard and my BF16 implementation. Numbers like $\frac{1}{3}$ and $\frac{21}{7}$ are particularly interesting because they are infinitely repeating decimals which cannot be stored precisely in 16 bits.

3.4 Phase 5: Hardware Implementation

Developing an accurate software implementation will simplify a seamless transition into a hardware design. Hardware description languages (HDL) are tools that use a syntax similar to C++ and are used to program the hardware on FPGAs. By achieving a nearly 1:1 translation from the functional C++ implementation into HDL, the BF16 implementation on the Vortex GPU will be finalized.

3.5 Phase 6: Performance Profiling & Analysis

The final step of this research project will be completing performance testing on BF16 hardware to see what kind of performance gains can be achieved. To make a fair comparison, I will write the same test program for both IEEE-754 and BF16. Then, I will compare metrics such as completion time, memory usage, and accuracy. I expect to see that BF16 reduces code size and speeds up computation without sacrificing too much accuracy.

4 Further Extensions & Impact

The primary objective of this project is to apply novel research to practical tasks like machine learning or artificial intelligence. The primary observation here is that performance improvements come at a trade-off. In this case, less information can be saved in 16 bits, rather than 32 bits. However, in the bigger picture the precision trade-off could prove to be negligible. If BF16 proves to be useful, there could be useful applications of it on low powered devices or even our own personal tasks. If BF16 proves to be insufficient for some tasks, there provides an opportunity to add support for mixed precision computing. Mixed precision computing allows for floating point formats to be switched on the fly, based on the task at hand [5].

References

- [1] ARM Technical Writing Staff. *What is Heterogenous Compute?* URL: <https://www.arm.com/glossary/heterogenous-compute>.
- [2] BRIAN CAULFIELD. *What's the Difference Between a CPU and a GPU?* URL: <https://blogs.nvidia.com/blog/2009/12/16/whats-the-difference-between-a-cpu-and-a-gpu/>.
- [3] William Kahan. *IEEE Standard for Binary Floating-Point Arithmetic (IEEE 754-1985)*. 1985.
- [4] Pankaj Kanwar. *BFloat16: The secret to high performance on Cloud TPUs*. URL: <https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus>.
- [5] *Train With Mixed Precision*. URL: <https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html>.